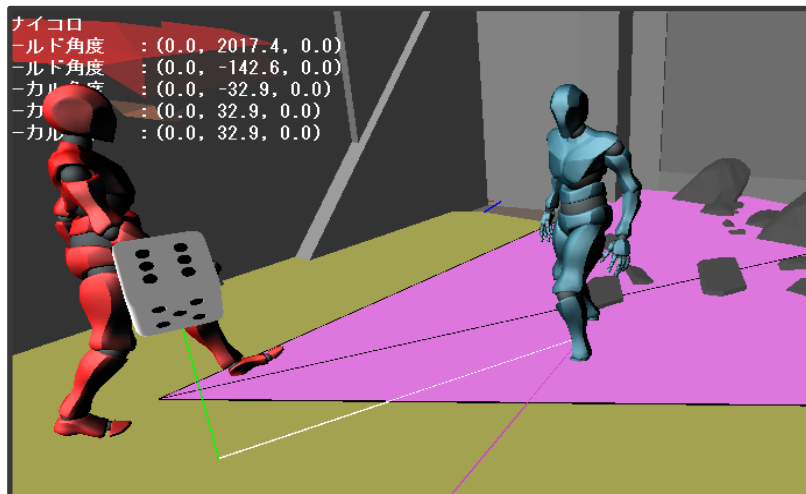


円錐型視野

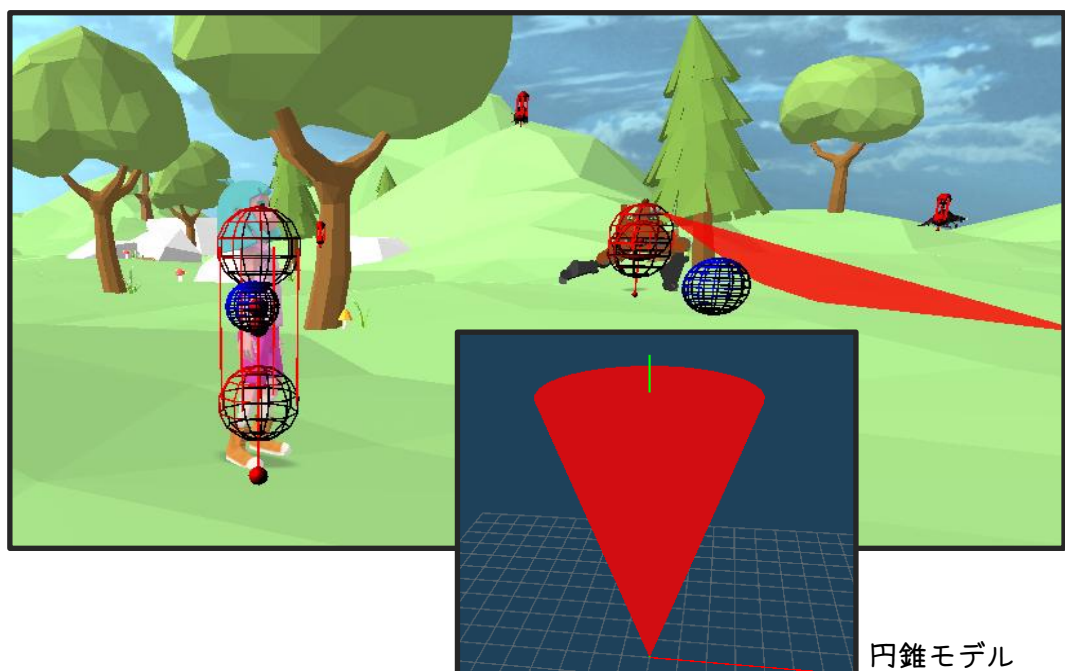
敵の行動パターンとして、“巡回”中にプレイヤーがNPCの視野に入ると、“警戒”状態に移行する流れを作りたいと思いますが、その前に、NPCの視野を表示させていきます。

3DWorldで実装した、三角形視野だと、ビジュアル的にあまりカッコよくありませんので、円錐モデルを使用した円錐視野を描画していきます。

三角形視野



円錐視野



EnemyRobot.h

```
class EnemyRobot : public EnemyBase
{
```

～ 省略 ～

private:

～ 省略 ～

// 巡回時の視野モデルの大きさ

static constexpr VECTOR VIEW_RANGE_SCL = { 8.0f, 4.0f, 2.0f };

// 巡回時の視野モデルの傾きX

static constexpr float VIEW_RANGE_ROT_X = 26.0f * DX_PI_F / 180.0f;

static constexpr float VIEW_RANGE_LOCAL_ROT_X = 90.0f * DX_PI_F / 180.0f;

// 巡回時の視野モデル位置同期用フレーム番号

static constexpr int VIEW_RANGE_SYNC_FRAME_IDX = 6;

// 巡回時の視野の広さ

static constexpr float VIEW_RANGE_PATROL = 600.0f;

// 巡回時の視野角

static constexpr float VIEW_ANGLE_PATROL = 16.0f;

～ 省略 ～

// 視野範囲用トランスフォーム

Transform viewRangeTransform_;

EnemyRobot.cpp

```
void EnemyRobot::Draw(void)
{
```

// 基底クラスの描画処理

CharactorBase::Draw();

```
#pragma region 視野(円錐)の描画
```

```
    SetUseLighting (FALSE);  
    MVIDrawModel (viewRangeTransform_. modelId);  
    SetUseLighting (TRUE);
```

```
#pragma endregion
```

```
    ~ 省略 ~
```

```
}
```

```
void EnemyRobot::InitLoad(void)  
{
```

```
    // 基底クラスのリソースロード  
    CharactorBase::InitLoad();
```

```
    // ロボットのモデルのロード  
    transform_. SetModel (  
        resMng_. LoadModelDuplicate (ResourceManager::SRC::ENEMY_ROBOT));
```

```
    // 視野(円錐)モデルのロード  
    viewRangeTransform_. SetModel (  
        resMng_. LoadModelDuplicate (ResourceManager::SRC::VIEW_RANGE));
```

```
}
```

```
void EnemyRobot::InitTransform(void)  
{
```

```
    // ロボット自身  
    transform_. scl = VScale (AsoUtility::VECTOR_ONE, SCALE);  
    transform_. quaRot = Quaternion();  
    transform_. quaRotLocal = Quaternion::Euler (DEFAULT_LOCAL_ROT);  
    transform_. Update();
```

```
    // 視野用円錐モデル  
    viewRangeTransform_. scl = VIEW_RANGE_SCL;  
    viewRangeTransform_. pos =  
        MVIGetFramePosition(transform_. modelId, VIEW_RANGE_SYNC_FRAME_IDX);
```

```

viewRangeTransform_. quaRot =
    transform_. quaRot. Mult(
        Quaternion::AngleAxis(VIEW_RANGE_ROT_X, AsoUtility::AXIS_X));
viewRangeTransform_. quaRotLocal =
    Quaternion::AngleAxis(VIEW_RANGE_LOCAL_ROT_X, AsoUtility::AXIS_X);
viewRangeTransform_. Update();

}

void EnemyRobot::UpdateProcessPost(void)
{

    EnemyBase::UpdateProcessPost();

    // 視野用円錐モデル同期
    ○○○

}

```

【要件①】

プログラムを完成させ、NPCから正しく視野が描画されること。

- ・ 円錐モデルの描画位置が気に入らなければ、位置を変えてOKです

【目標①】

NPCの視野範囲がゲームユーザとして正しく認識できること。

【要件②】

プレイヤーとNPCの視野を衝突判定させて、NPCをAlert状態へ遷移させること。

- Alert遷移後は、以下の行動を取ることに

EnemyRobot.cpp

```
void EnemyRobot::ChangeStateAlert(void)
{

    stateUpdate_ = std::bind(&EnemyRobot::UpdateAlert, this);

    // 移動量ゼロ
    movePow_ = AsoUtility::VECTOR_ZERO;

    // ダンス(足踏み)アニメーション再生
    animationController_>Play(
        static_cast<int>(ANIM_TYPE::DANCE), true);

}
```

- 設計に関しては、特に縛りはない

その上で、

EnemyManager や EnemyBase にPlayerクラスのポインタを渡しても良い

- 視野は円錐モデルではあるが、3DWorldのように、三角と座標の衝突判定でも良い
(精度荒い)
- 視野の3Dモデルと、カプセルとの衝突判定でも良い
(精度高め)
- 頑張って、視野の3Dモデルを使用せず、ランタイム上で円錐を作成し、円錐とカプセルの衝突判定でも良い
(精度高め)

【目標②】

プレイヤーが視野範囲に入ると、
NPCが足踏み(ダンス)アニメーションを行うこと。

